Database Design for TeamMatch

Introduction

The TeamMatch database is designed to store information about students, courses, projects, and teams so the system can automatically form balanced groups. The database organizes user profiles, skills, preferences, and team assignments in a structured way. Each table has a primary key to uniquely identify records, and foreign keys are used to connect related data between tables.

This design separates different types of information into their own entities to avoid duplication and improve efficiency. For example, skills are stored separately from users, and team membership is stored in its own table to handle many-to-many relationships. Overall, this structure supports the required system features such as student matching, team creation, and project organization.

Entities and Attributes

User

- user_id (PK)
- first_name
- last_name
- email
- password_hash
- role
- created_at

Course

- course_id (PK)
- course_code
- course_name
- semester
- instructor_name

Team

- team_id (PK)
- team_name

- course_id (FK)
- created_at
- status

Student_Profile

- profile_id (PK)
- user_id (FK)
- major
- skill_level
- interests
- availability
- preferred_role
- bio

Project

- project_id (PK)
- course_id (FK)
- title
- description
- required_team_size
- deadline
- status

Team_Membership

- membership_id (PK)
- team_id (FK)
- user_id (FK)
- joined_at
- member_role

Match_Request

- request_id (PK)
- user_id (FK)
- project_id (FK)
- preferred_team_size
- preferences
- submitted_at
- status

Match_Result

- match_id (PK)
- request_id (FK)
- team_id (FK)
- compatibility_score
- generated_at

Skill

- skill_id (PK)
- skill_name
- category

User_Skill

- user_skill_id (PK)
- user_id (FK)
- skill_id (FK)
- proficiency_level

Primary Keys

The primary keys used in the database are:

- user_id in User
- course_id in Course
- team_id in Team
- profile_id in Student_Profile
- project_id in Project
- membership_id in Team_Membership
- request_id in Match_Request
- match_id in Match_Result
- skill_id in Skill
- user_skill_id in User_Skill

Foreign Keys

The foreign keys connect related tables in the database:

- Team.course_id references Course.course_id
- Student_Profile.user_id references User.user_id

- Project.course_id references Course.course_id
- Team_Membership.team_id references Team.team_id
- Team_Membership.user_id references User.user_id
- Match_Request.user_id references User.user_id
- Match_Request.project_id references Project.project_id
- Match_Result.request_id references Match_Request.request_id
- Match_Result.team_id references Team.team_id
- User_Skill.user_id references User.user_id
- User_Skill.skill_id references Skill.skill_id

Relationships

The relationships in the TeamMatch database include:

- One Course can have many Teams, which is a one-to-many relationship (1:N)
- One Course can have many Projects, which is a one-to-many relationship (1:N)
- One User has one Student_Profile, which is a one-to-one relationship (1:1)
- One Team can have many Team_Membership records, which is a one-to-many relationship (1:N)
- One User can have many Team_Membership records, which is a one-to-many relationship (1:N)
- One User can submit many Match_Requests, which is a one-to-many relationship (1:N)
- One Project can receive many Match_Requests, which is a one-to-many relationship (1:N)
- One Match_Request generates one Match_Result, which is a one-to-one relationship (1:1)
- One Team can appear in many Match_Results, which is a one-to-many relationship (1:N)
- Users and Skills have a many-to-many relationship (M:N) resolved using User_Skill
- Users and Teams have a many-to-many relationship (M:N) resolved using Team_Membership

---

Design Explanation

This database design supports the main features of the TeamMatch system. The User and Student_Profile tables store student information such as skills, interests, and availability. The Course and Project tables allow instructors to create projects within a specific class. The Team and Team_Membership tables store team assignments and allow multiple students to be grouped together.

The Match_Request and Match_Result tables support the team matching functionality by storing user preferences and generated team results. The Skill and User_Skill tables are used to store student skills without duplicating data. This structure keeps the data organized, reduces redundancy, and allows the system to efficiently create and manage student teams.

Conclusion

This database design organizes the data needed for the TeamMatch system and defines clear relationships between entities. Primary keys uniquely identify each record, and foreign keys maintain connections between related tables. The design supports student profiles, project management, team formation, and matching functionality. The ER diagram visually shows how all entities are connected and how data flows through the system.